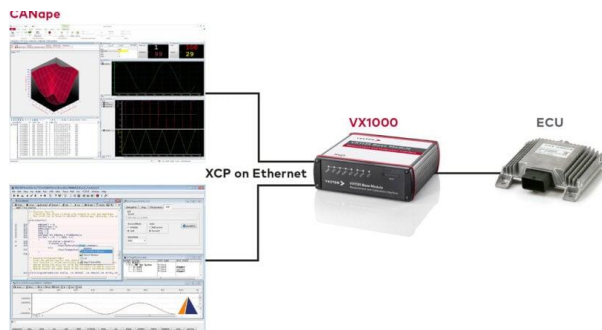


1. XCP signifies “Universal Measurement and Calibration Protocol” .

The “X” stands for the variable and interchangeable transport layer.
XCP is a higher level protocol used for communication between a measurement and calibration system(i.e. CANape) and an electronic control unit(ECU).

1.1. General Introduction to XCP at Powertrain:

- <https://volvogroup.sharepoint.com/sites/unit-deep/SitePages/XCP---Calibration-Protocol---Why-is-it-so-important-in-Automotive-.aspx> (Presenter Andreas Henning)



2. Introduction:

- The primary applications of XCP are measurement and calibration of internal ECU parameters
- XCP standardized by an [ASAM](#) working committee ([Wikipedia: Association for Standardisation of Automation and Measuring Systems](#)). ASAM is an organization of automotive OEMs, suppliers and tool producers.
- XCP protocol succeeds CCP (CAN Calibration Protocol).
- XCP is based on the Master-Slave principle.

3. Difference between CCP and XCP Protocol:

XCP is a generalization of CCP with clear separation between transport layer and protocol layer. The conceptual idea of the CAN Calibration Protocol was to permit read and write access to internal ECU data over CAN, its successor, XCP

was developed to implement this capability via different transmission media i.e. XCP on CAN, XCP on FlexRay or XCP on Ethernet.

4. Mode of operation:

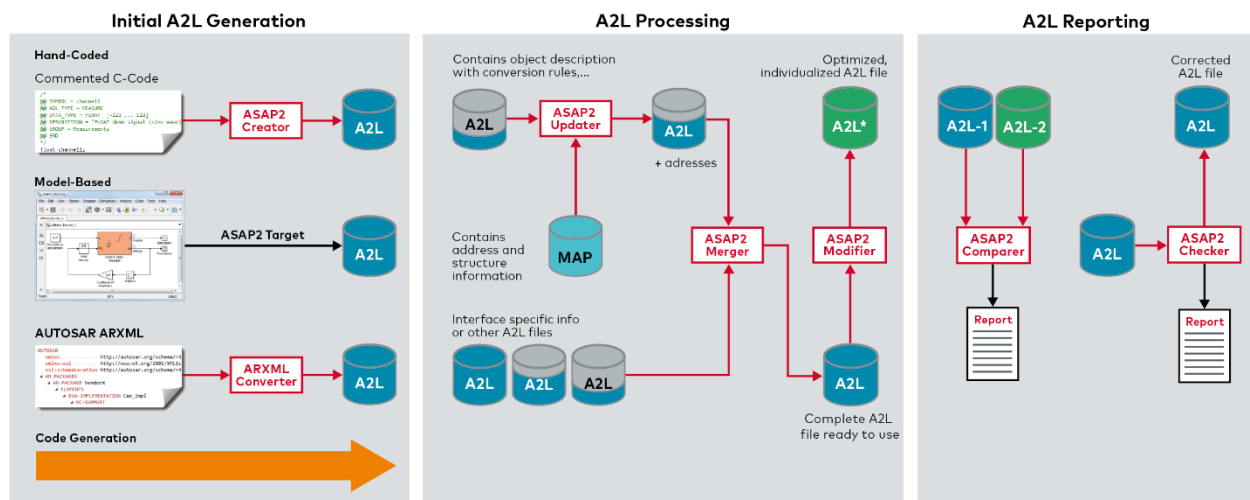
- XCP is designed as a Single Master/Multi Slave system. The ECU is the Slave and the measurement and calibration tool is the Master. A Slave may only communicate with one Master at any given time; on the other hand, the Master can simultaneously communicate with many Slaves.
- An example situation could be in a vehicle where a single master is connected to the ECU handling engine control as well as another ECU controlling adaptive suspension. The master controller is then able to measure internal controller states, as well as tune parameters for the embedded system while evaluating

performance

5. Memory Fundamentals of the XCP Protocol:

- A key functionality of XCP is that **it enables read and write access to the memory of the Slave.**
- Read access lets users measure the time response of an internal ECU parameter.
- The accesses are **address-oriented**, i.e. the communication between Master and Slave references addresses in memory.
- The **measurement of a parameter** is essentially implemented as a **request of the Master to the Slave**: “to get the value of memory location 0x1234” .

- The **Calibration of a parameter** - the write access - to the Slave means: “Set the value at address 0x9876 to 5” .
- In order to modify individual parameters at runtime in the ECU, there must be access to them. Not every type of memory permits this process, often only RAM or NVM emulating EEPROM support multiple write.
- **Flash memories** are used for long-term storage of code and data, even without power supply. The special aspect of a flash memory is that read and write access to individual bytes is indeed possible at any time, but erasure of contents can only be done blockwise, usually in rather large blocks. hence already written values which are located in the flash area cannot be modified at runtime without special erasure operations. If a change to parameter values should be possible during runtime, the parameters to be modified must lie in RAM and not in flash.
- **A2L file creation:** The relationship between object name, data type and address is needed, but as the compiler link file normally is not enough, is the ELF file for debuggers used for extraction of type and addresses; In Vector documentation are all alternatives named as “MAP file”, see Vector overview of their [asap2-tool-set](#)



6. XCP message frame:

- The entire “**XCP message frame**” is embedded in a frame of the transport layer. The frame consists of three parts: the **XCP header**, the **XCP packet** and the **XCP tail**. Red part of a message is used to send the current XCP frame. The XCP header and XCP tail depend on the transport protocol.
- XCP packet contains three components: “**Identification Field**”, “**Timestamp Field**” and “**Data Field**”. Each Identification Field begins with the Packet Identifier (PID), which identifies the packet.
- The type of communication via XCP packets is subdivided into **one range for commands** (CTO) and **one range for sending synchronous data** (DTO).
- Commands are exchanged via **CTOs** (Command Transfer Objects). The Slave must always respond to a CMD with RES or ERR. The other CTO messages are sent asynchronously. The Data Transfer Objects (**DTO**) are used to exchange synchronous measurement and stimulation data.
- XCP always defines addresses with five bytes: four for the **actual address** and **one byte for the address extension**. Based on a **CAN transmission**, only **seven useful bytes** are available for XCP messages. For example, if the calibrator sets a 4-byte value and wants to send both pieces of information in one CAN message, there is insufficient space to do this. Since a total of nine bytes are needed to transmit the address and the new value, the change cannot be transmitted in one CAN message (seven useful bytes). The calibration request is therefore made with two messages from Master to Slave. The Slave must acknowledge both messages and in sum four messages are exchanged.

Identification Field:

When messages are exchanged, both the Master and Slave must be able to determine which message was sent by the other. This is accomplished in the identification field. That is why each message begins with the Packet Identifier (PID).

Timestamp:

DT0 packets use timestamps, but this is not possible in transmission of a CTO message. The Slave uses the timestamp to supply time information with measured values. That is, the Master not only has the measured value, but also the time point at which the measured value was acquired. Transmission of a timestamp from the Slave is optional.

Data Field:

The XCP packet also contains the data stored in the data field. In the case of CTO packets, the data field consists of specific parameters for the different commands. DT0 packets contain the measured values from the Slave.

7. XCP Command execution :

- The Slave receives a command from the Master and must react to it with a positive or negative Response.
- At the start of a communication between Master and Slave, the Master sends a connect request to the Slave, which in turn must respond positively to produce a continuous point-to-point

connection.

Depicts the basic flow of command execution of the core XCP protocol

8. Example of an XCP Implementation:

The XCP protocol layer is placed over the XCP transport layer.

The software interface between the transport and protocol layers:

- When the Slave receives an XCP message over the bus, it first arrives in the **communication driver**, which routes the message to the XCP transport layer.

- The transport layer informs the protocol layer about the message with the function call `XcpCommand()`.
- If the XCP protocol layer wishes to send a message (e.g. a **response** to an XCP command from the Master or a DAQ message), the message is routed to the transport layer by a call of the `ApplXcpSend()` function.
- The transport layer informs the protocol layer that the message was successfully sent by the function call `XcpSendCallback()`.

The interface between the application and the protocol layer:

- The application activates the XCP driver with the help of `XcpInit()`. This call is made once in the starting process.
- `XcpEvent()`, the application informs the XCP driver that a certain event has occurred (e.g. “End of a computational cycle reached”).
- `XcpBackground()` lets the XCP driver execute certain activities in background (e.g. calculation of a checksum).
- the XCP driver uses the function `ApplXcpGetPointer()` to obtain a pointer from a A2L-conformant address. Since the addresses in A2L files are always defined as 40-bit values (32-bit address, 8-bit address extension).
- Parameters which are needed to be calibrated runtime should be stored in RAM. So, in order to calibrate the parameters the master sends a set of commands to the slave (ECU). The slave responds back with the results.
- Exploring the memory allocation for XCP parameters is done on VCM ECU. So the screenshots attached for reference are tried out/ fetched out from VCM configuration.
- **4.1 Memory Section for XCP parameters:**
 - All the parameters whose values can be updated during run time are stored in RAM. Initialization is done while the ECU is getting booted. So all the parameters which are stored in the RAM are initialized to their initial values.
 - **4.1.1 Hardware region in vBaseEnv**
 - Create a hardware region in vBaseEnv stating the start address and the size.
 - **NOTE:** Value mentioned in screenshot (Figure 1) is just for reference.

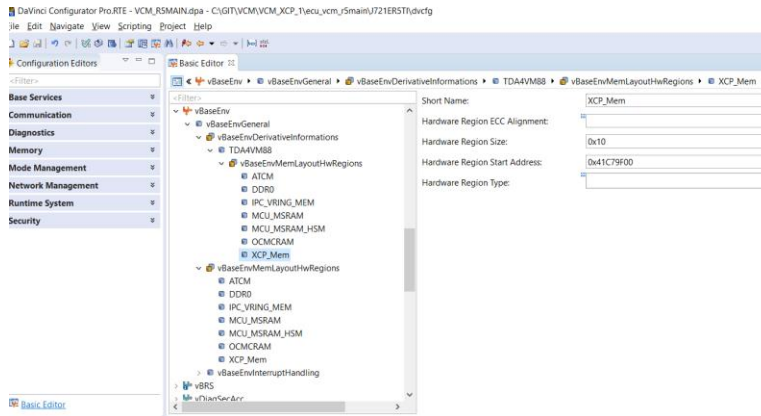
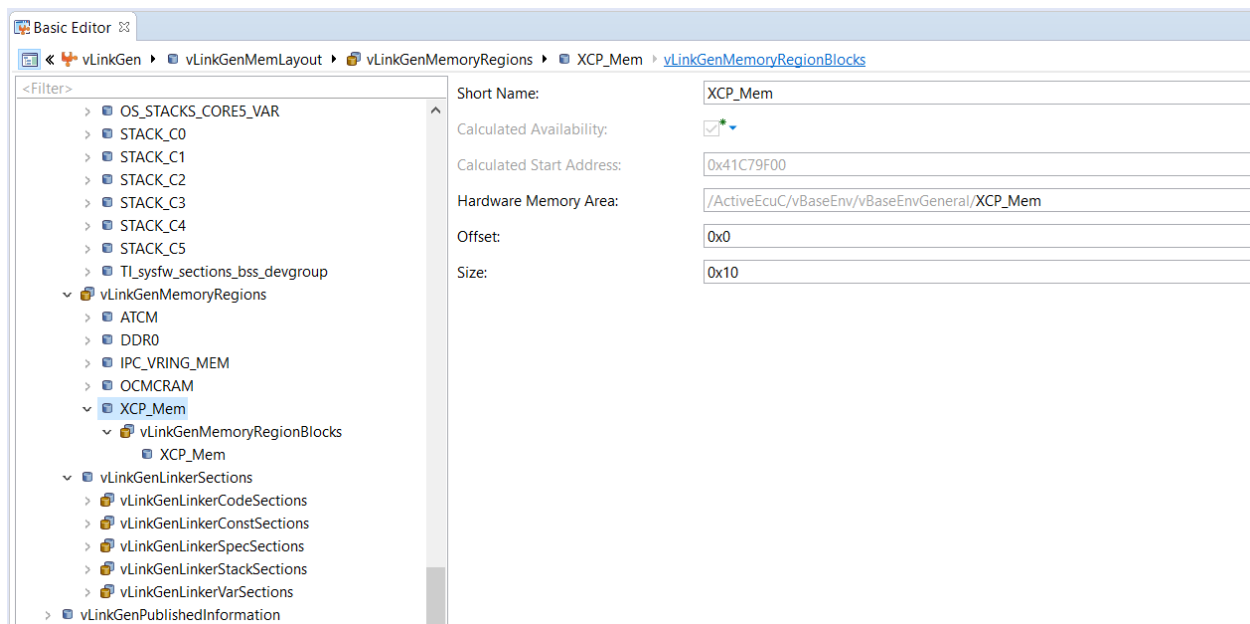


Figure 1

- 4.1.2 LinkGen Memory Region:



- Create a memory region and target the same to respective Hardware memory area as reference (Figure 3 and Figure 4)

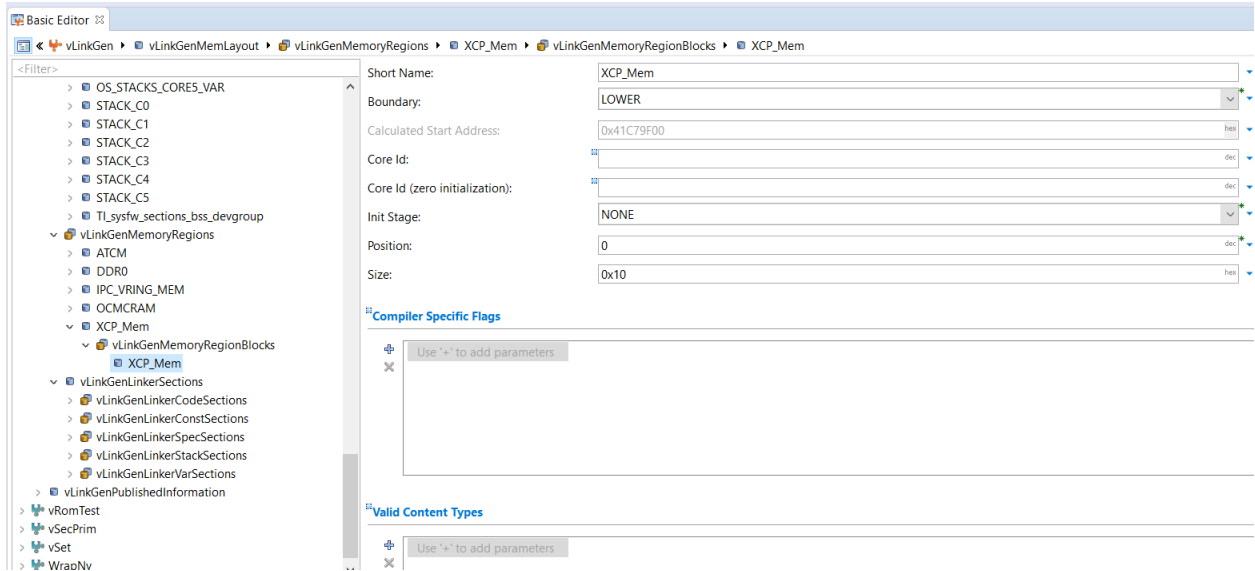


Figure 3

-
- Figure 4
-
- 4.1.3 Variable section group:
- Create a section group and target the same to respective hardware memory region created (Figure 5 and Figure 6).

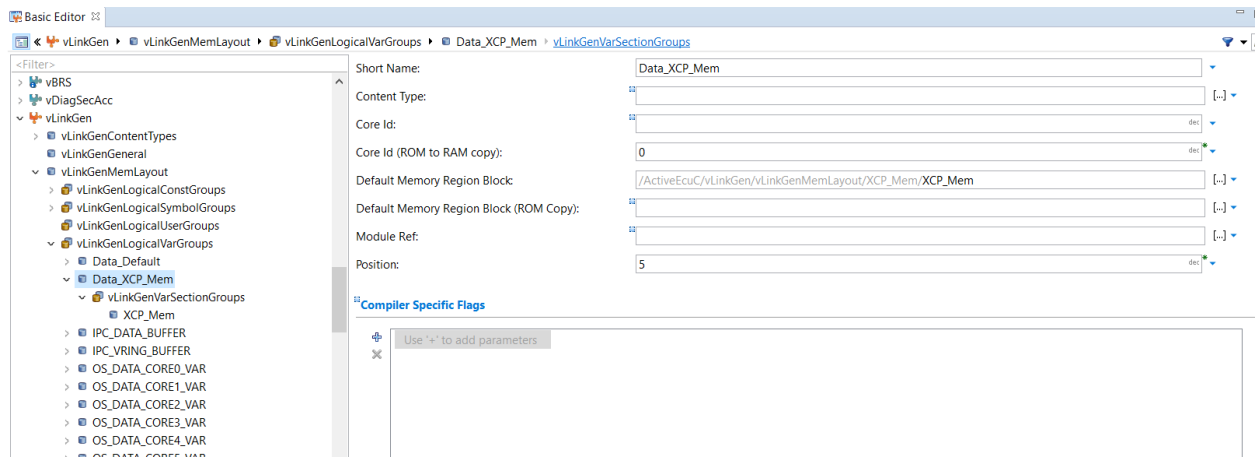
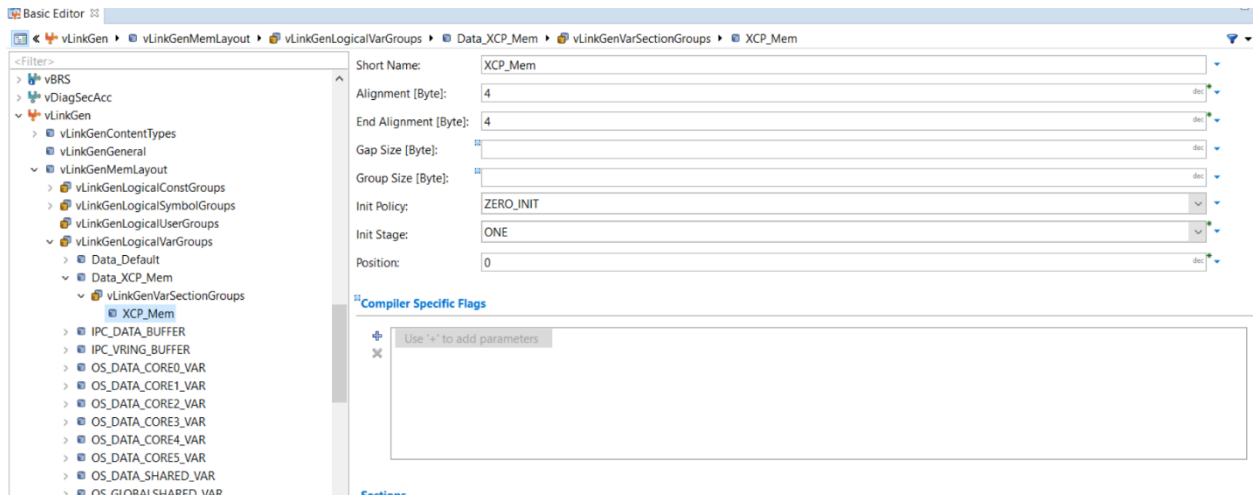
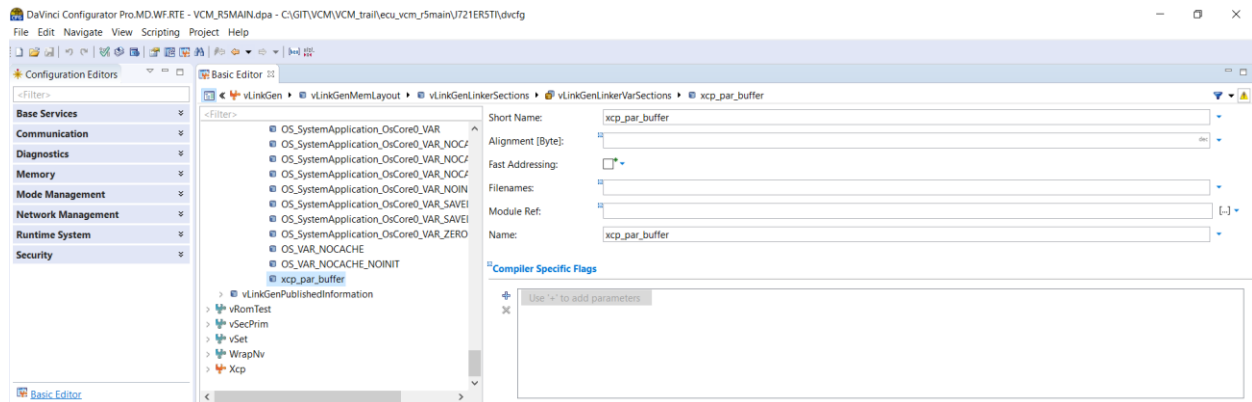


Figure 5

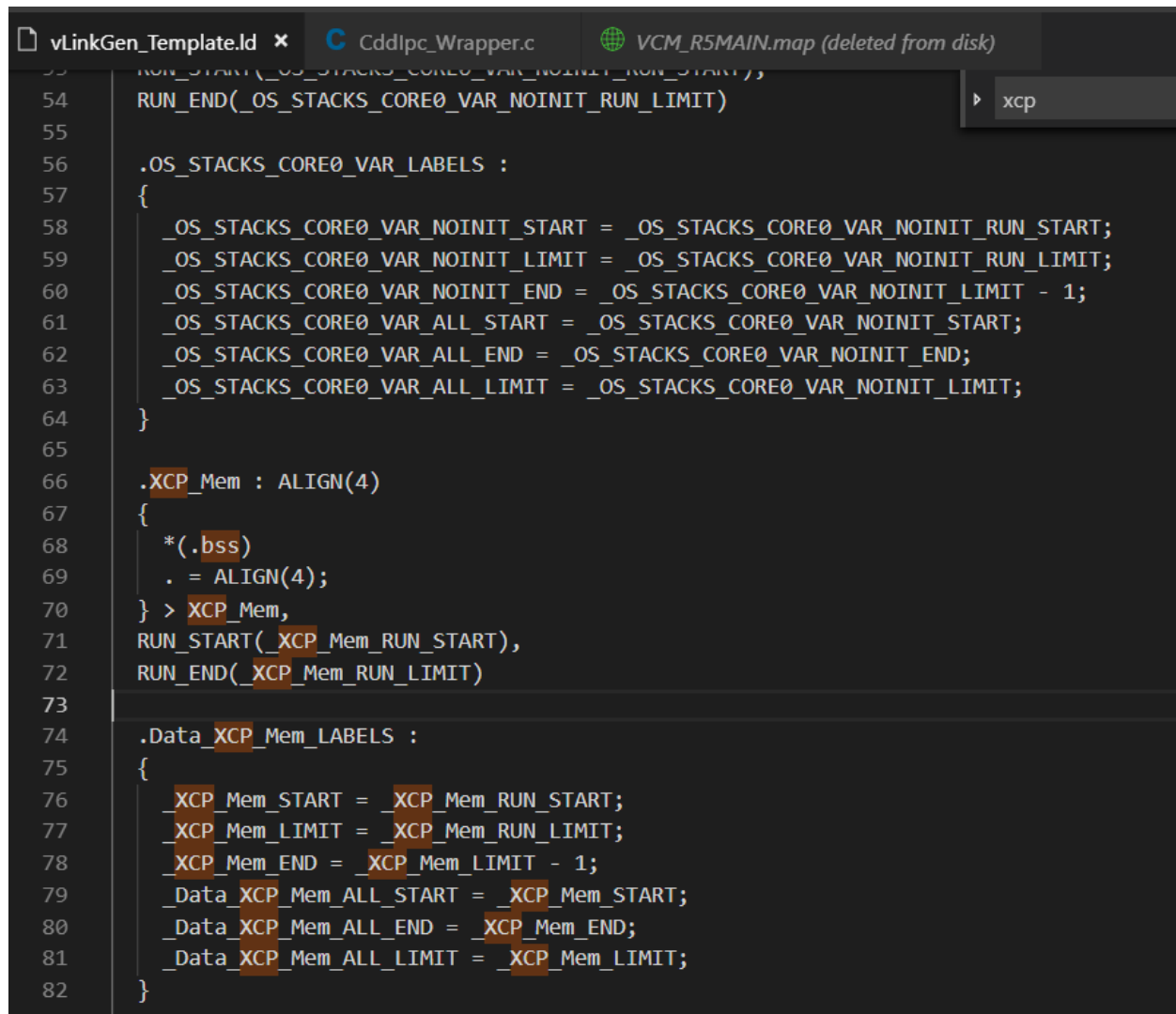
-



-
-
- **4.1.3 Linker variable section:**
- Create a linker variable section and map this section to linker group



-
- **4.1.5 Linker file:**
- Generated linker file will be as below. Same section will be used as a reference to make the parameters to use a specific memory location.



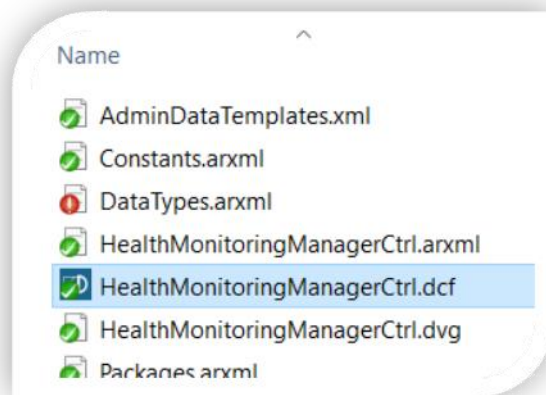
The screenshot shows a code editor with three tabs: 'vLinkGen_Template.ld', 'CddIpc_Wrapper.c', and 'VCM_R5MAIN.map (deleted from disk)'. The active tab is 'vLinkGen_Template.ld', which contains assembly code. A search bar on the right shows 'xcp'. The code includes sections for OS stacks, XCP memory alignment, and data labels.

```
53 RUN_START(_OS_STACKS_CORE0_VAR_NOINIT_RUN_START);
54 RUN_END(_OS_STACKS_CORE0_VAR_NOINIT_RUN_LIMIT)
55
56 .OS_STACKS_CORE0_VAR_LABELS :
57 {
58     _OS_STACKS_CORE0_VAR_NOINIT_START = _OS_STACKS_CORE0_VAR_NOINIT_RUN_START;
59     _OS_STACKS_CORE0_VAR_NOINIT_LIMIT = _OS_STACKS_CORE0_VAR_NOINIT_RUN_LIMIT;
60     _OS_STACKS_CORE0_VAR_NOINIT_END = _OS_STACKS_CORE0_VAR_NOINIT_LIMIT - 1;
61     _OS_STACKS_CORE0_VAR_ALL_START = _OS_STACKS_CORE0_VAR_NOINIT_START;
62     _OS_STACKS_CORE0_VAR_ALL_END = _OS_STACKS_CORE0_VAR_NOINIT_END;
63     _OS_STACKS_CORE0_VAR_ALL_LIMIT = _OS_STACKS_CORE0_VAR_NOINIT_LIMIT;
64 }
65
66 .XCP_Mem : ALIGN(4)
67 {
68     *(.bss)
69     . = ALIGN(4);
70 } > XCP_Mem,
71 RUN_START(_XCP_Mem_RUN_START),
72 RUN_END(_XCP_Mem_RUN_LIMIT)
73
74 .Data_XCP_Mem_LABELS :
75 {
76     _XCP_Mem_START = _XCP_Mem_RUN_START;
77     _XCP_Mem_LIMIT = _XCP_Mem_RUN_LIMIT;
78     _XCP_Mem_END = _XCP_Mem_LIMIT - 1;
79     _Data_XCP_Mem_ALL_START = _XCP_Mem_START;
80     _Data_XCP_Mem_ALL_END = _XCP_Mem_END;
81     _Data_XCP_Mem_ALL_LIMIT = _XCP_Mem_LIMIT;
82 }
```

The measurement and calibration parameters of a SWC for the measurement and calibration task can be done by adjusting some setting or enabling a option in Davinci Developer. Please find those settings below,

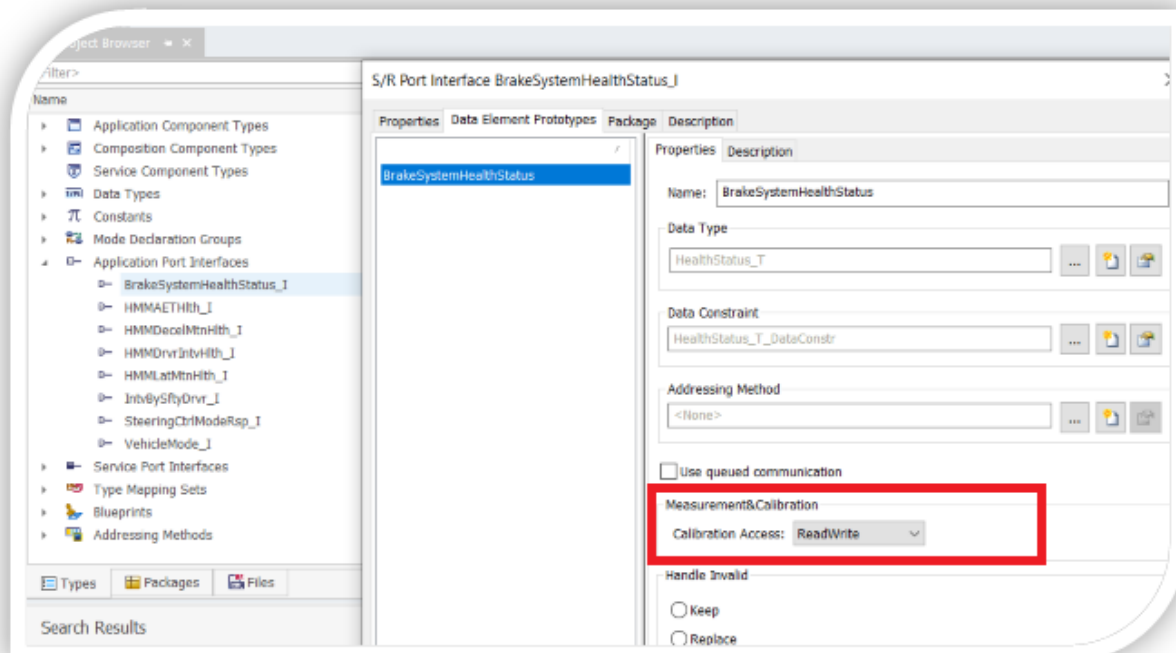
Step1:

Choose a SWC and open .dcf file of that particular SWC.



Step2:

Choose a port interface which you want to enable measurement and calibration support. Mention the access level as per your need and say apply&ok



NOTE:

Port interface with a system signal (COM) mapped will not be included in this feature (Yet the moment)

Step3:

Perform update dvcfg or sync changes of Developer and configurator to see the reflections of change made in step2

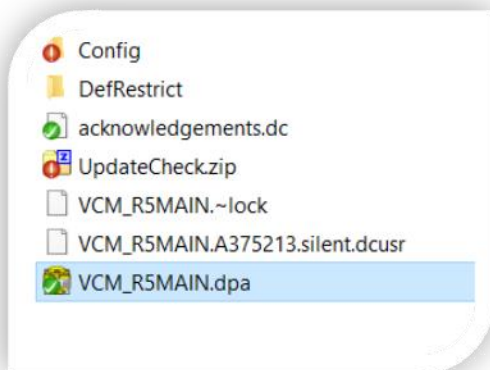
To generate a2l file with port interface details from Davinci Configurator needs some additional steps that to be done in the tool.

Prerequisite: Steps mentioned in page [Adaptions to Davinci Developer](#) has to be followed mandatorily. If not this feature is not fulfilled.

Please check the same below

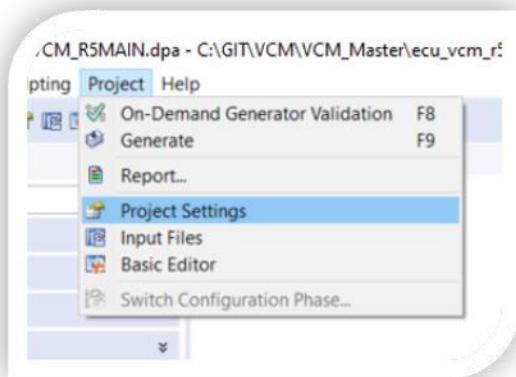
Step1:

Open dpa file of the respective project



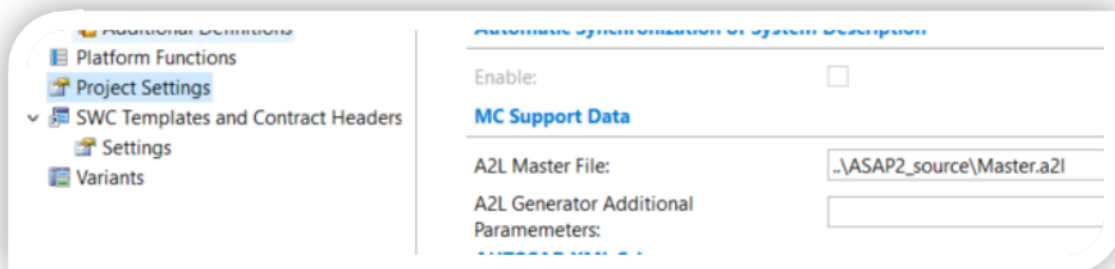
Step2:

Once dpa is opened, please go to project settings as shown

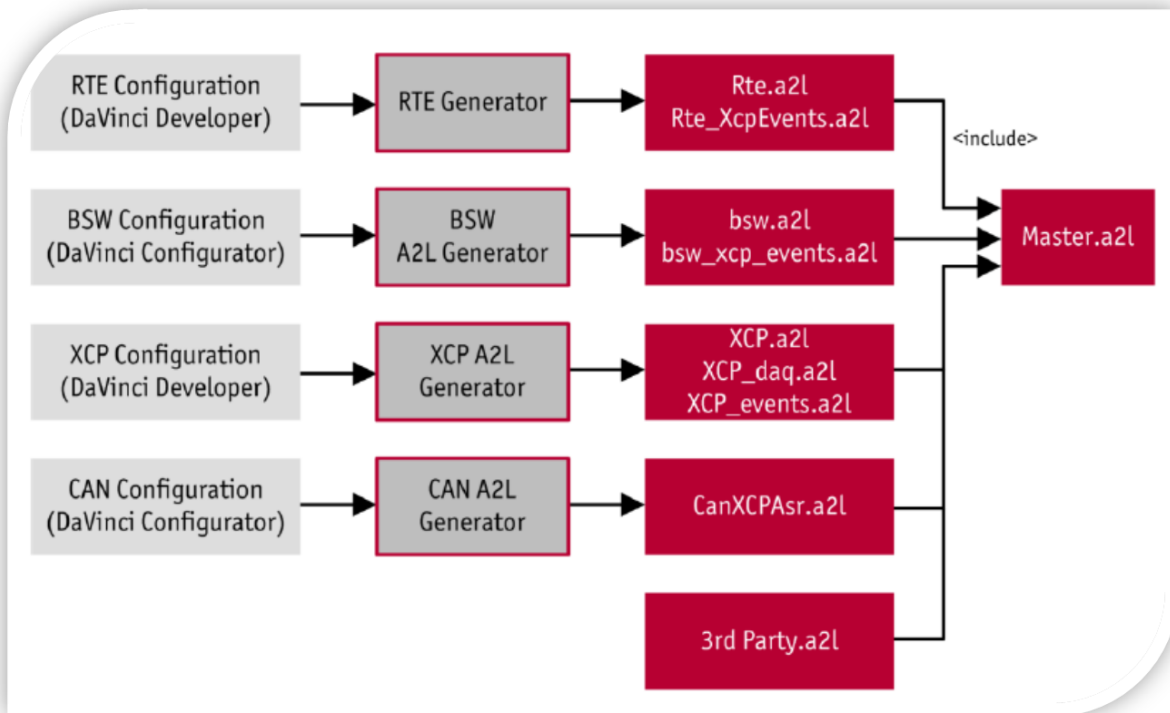


Step3:

In project settings, click on project settings option again and try to find a column with MC Support Data



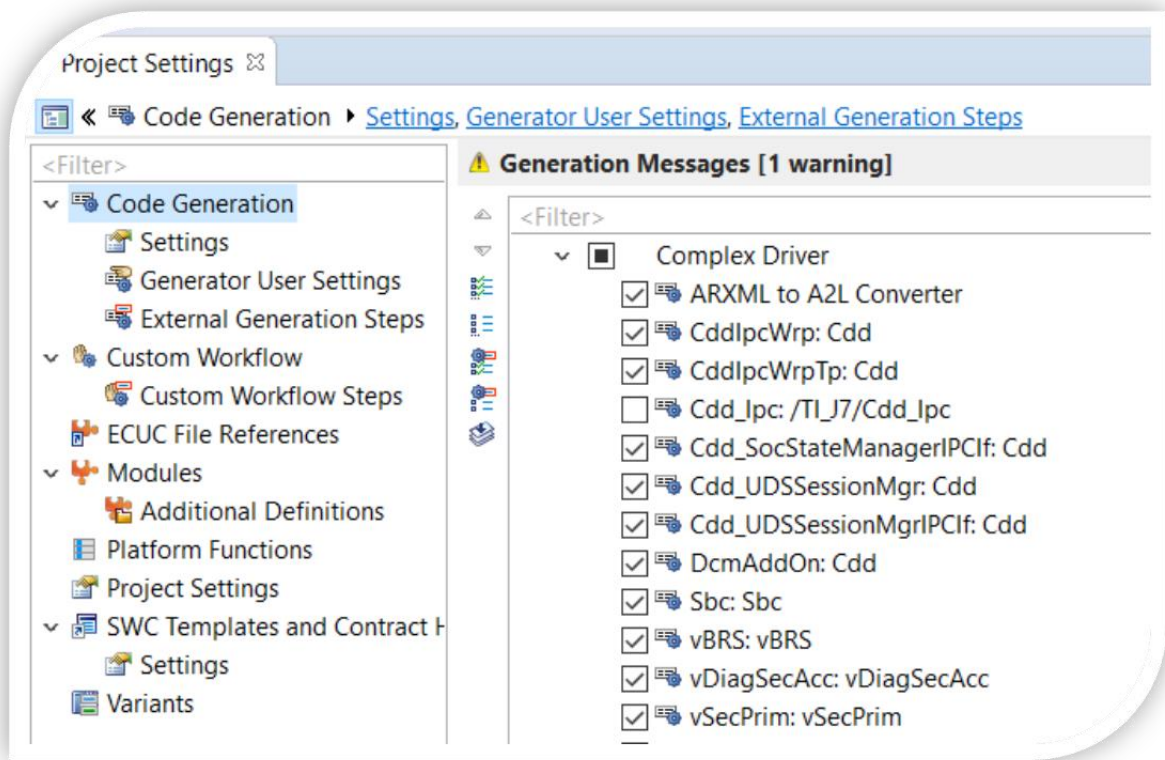
A master A2L file is a very project specific file that merges all partial databases into one is required. This master file can then be used as a template for the file to be created. The objective is to ultimately obtain a single file containing all information.



Note: MICROSAR XCP provides a _Master.a2l file as a template in the delivery folder ...\\<Delivery>\\Misc\\McData. Same file can be copied to the project folder and that reference is mentioned in here in the tool. Gere Master.a2l file has other a2l file inclusions as well but user is free to choose which a2l files are need and merged together.

Step4:

Please enable **ARXML to A2L converter** option to generate McData.a2l file that contains port interface information along with address parameters.



Step5:

Below pictures shows the details about generate folder structure, where user can find the generated output files.

Folder Structure	
Module Files:	..\derived\dvcfg
VTT Module Files:	..\derived\vtt_dvcfg
Template Files:	..\derived\template_dvcfg
Service Component Files:	.\Config\ServiceComponents
Bsw Internal Behavior Files:	.\Config\InternalBehavior
Log Files:	..\derived\log
Measurement and Calibration Files:	..\derived\McData
AUTOSAR Files:	.\Config\AUTOSAR

NOTE: The McSupportData is not generated by default in the Rte. Only after the McSupportData is exposed to the BSW Module Descriptions, which is done with the auto solving Action **RTE01239**. This option can be found in validation tab

⚙ RTE01239	McSupportData Flat instance descriptors out of sync (1 message)
⚙ RTE01239	FlatInstanceDescriptors need to be updated to allow creation of McSupportData
	🔧 Update FlatInstanceDescriptors for McSupportData generation
	🔗 ComponentType/ct/ct_InternalBehavior/PerInstanceMemory/@swDataDefProps<MISwDataDefProps>

Step6:

If step1 to 5 is followed, then user can give a generate code and can able to see all needed port interface options in a2l file

